TP2 — Construction d’une infrastructure de gestion de clés

1. Objectif du TP

Ce TP avait pour objectif de construire une infrastructure de gestion de clés (PKI) avec OpenSSL. Il s’agissait de créer une autorité de certification racine, une autorité intermédiaire et des certificats numériques, puis de vérifier leur validité et de mettre en œuvre leur révocation.

L’environnement de travail utilisé était Ubuntu avec OpenSSL.

2. Création et configuration des autorités de certification

2.1. Autorité racine

Nous avons créé une clé privée elliptique de courbe P-384, puis un certificat racine auto-signé valable 10 ans.

Ses principales extensions sont :

- `CA:TRUE,pathlen:1` : autorise la signature de certificats et une autorité intermédiaire.
- `keyCertSign,cRLSign` : autorise la signature de certificats et de listes de révocation.
- `subjectKeyIdentifier` : permet d'identifier la clé publique.

Le sujet et l’émetteur sont identiques, car le certificat racine est auto-signé. Sa clé privée doit rester hors ligne afin de limiter les risques de compromission.

2.2. Autorité intermédiaire

Une clé privée P-256 a été créée pour l’autorité intermédiaire. Sa demande de certificat (CSR) a ensuite été signée par l’autorité racine. Le certificat obtenu est valable 5 ans.

Ses principales extensions sont :

- `CA:TRUE,pathlen:0` : autorise la signature de certificats, mais interdit la création d’une autre autorité intermédiaire.
- `keyCertSign,cRLSign` : autorise la signature de certificats et de CRL.
- `nameConstraints` : limite les noms autorisés au domaine `tp.local`.

Les fichiers `racine.cnf` et `inter.cnf` contiennent les paramètres des autorités et les extensions utilisées pour émettre les certificats.

3. Émission et vérification des certificats

Deux certificats ont été créés et signés par l’autorité intermédiaire. Le premier est destiné au serveur `www.tp.local`, et le second à l’utilisateur `alice@tp.local`.

Les deux certificats utilisent des clés P-256 et ont une durée de validité de 90 jours. Le certificat serveur contient l'usage `serverAuth` et les noms DNS dans son extension SAN. Celui d'Alice contient l'usage `clientAuth` et son adresse électronique dans le SAN.

Les clés privées ont été générées séparément pour chaque titulaire afin qu'elles restent sous leur contrôle et ne soient pas transmises à l'autorité de certification.

Les vérifications réalisées avec OpenSSL ont donné les résultats suivants :

- La chaîne de certification du serveur a été validée : `serveur.crt: OK`.
- Le nom d'hôte `www.tp.local` a été accepté.
- Un nom d'hôte incorrect a été rejeté.
- Le certificat d'Alice a été validé avec l'option `-purpose sslclient`.

Un certificat destiné à `www.example.com` a également été émis, mais sa validation a échoué à cause des contraintes de noms. Cela montre que l'émission et la validation d'un certificat sont deux opérations distinctes : la validation applique les contraintes définies dans la chaîne de certification.

4. Révocation des certificats

La révocation permet d'invalider un certificat avant la fin de sa période de validité, notamment en cas de compromission de sa clé privée.

Au cours du TP, le certificat d'Alice a été révoqué avec la raison `keyCompromise`. Une CRL a ensuite été générée par l'autorité intermédiaire. L'ancien certificat intermédiaire a également été révoqué par l'autorité racine avec la raison `superseded`.

Les tests ont confirmé que :

- Le certificat révoqué d'Alice était refusé.
- L'ancien certificat intermédiaire était refusé.
- Le certificat serveur encore valide était accepté.

Les CRL permettent de communiquer la liste des certificats révoqués, mais elles peuvent être volumineuses et ne sont pas toujours actualisées immédiatement. OCSP permet de consulter le statut d'un certificat, tandis que les certificats de courte durée limitent la période d'exploitation possible en cas de compromission.

5. Automatisation et documentation

Le script `pki.sh` a été réalisé pour automatiser les étapes de création de l'autorité racine, de l'autorité intermédiaire et des certificats. Il utilise `set -euo pipefail` afin d'interrompre son exécution en cas d'erreur. Sa syntaxe a été vérifiée avec `bash -n`.

Les questions de réflexion rencontrées pendant le TP ont été traitées dans le fichier `README.md` fourni avec le projet.

6. Conclusion

Ce TP a permis de mettre en pratique les principales opérations d’une infrastructure PKI : création des autorités de certification, émission et vérification des certificats, gestion des contraintes de noms et révocation.

Les résultats obtenus confirment le bon fonctionnement de la chaîne de certification et le rejet des certificats non conformes ou révoqués. Le script réalisé permet également d’automatiser une partie des opérations.